Bazele inteligenței artificiale

Laborator – Grigore Ionuț

Reguli laborator

NOTA FINALA BIA = (NOTA LAB + NOTA EXAMEN)/2

Regulile sunt următoarele:

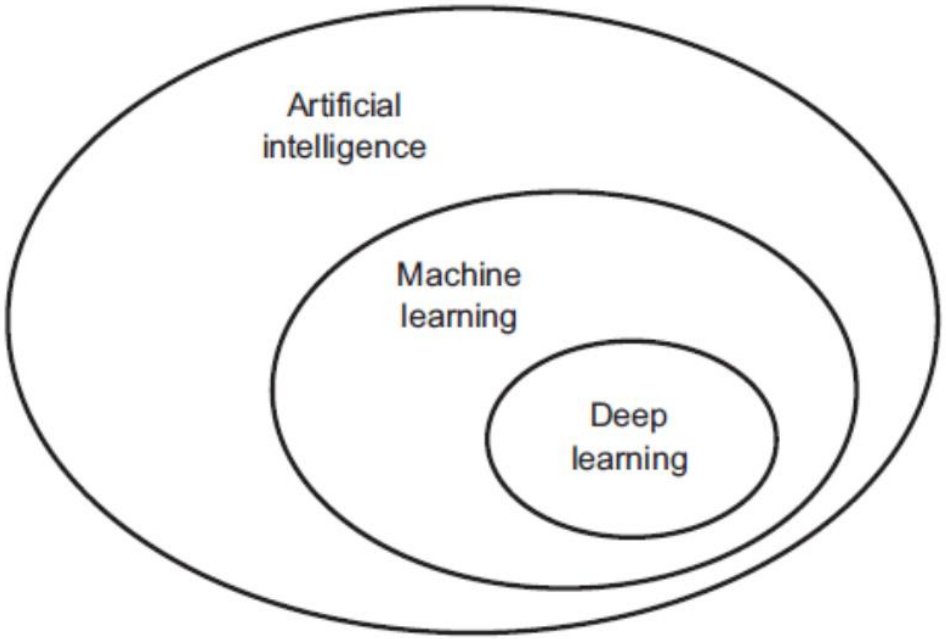
1. Vom avea 13 laboratoare
2. Maxim 2 absențe, recuperate în S14.
3. NOTA LAB = (T1 + T2)/2; T1 → S7; T2 → S13
4. T1 → materia din săptămânile 1-6, iar T2 → materia din săptămânile 8-12.
5. Nu se permit măriri ale notelor de la teste ☹ . Testele care nu au fost date, vor fi recuperate în S14.

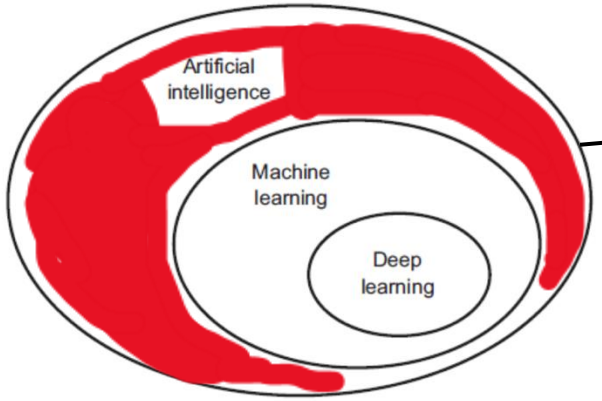
Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

ML

DL

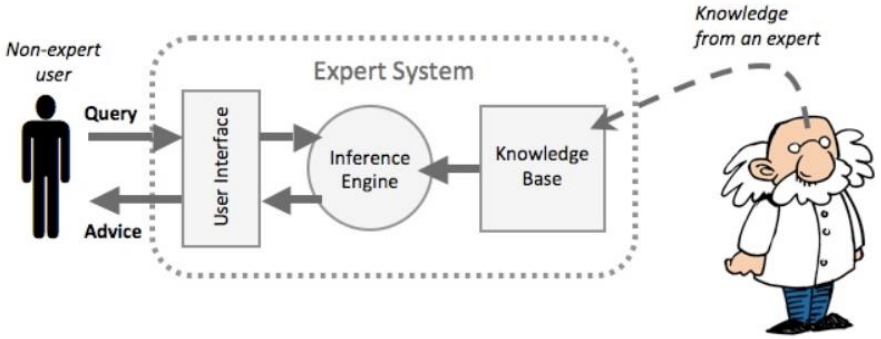




does not involve learning from data !

Example of problems:

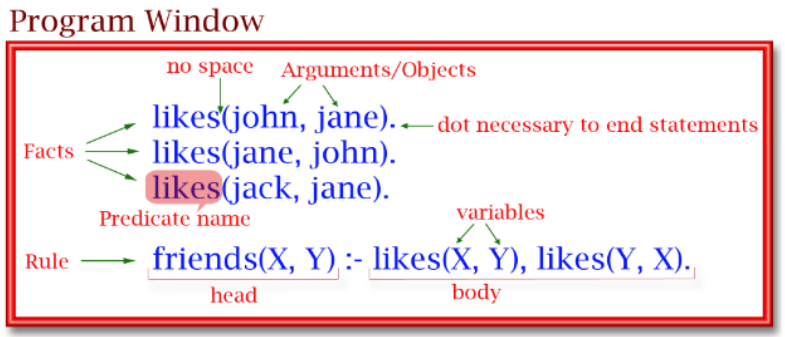
Expert systems:

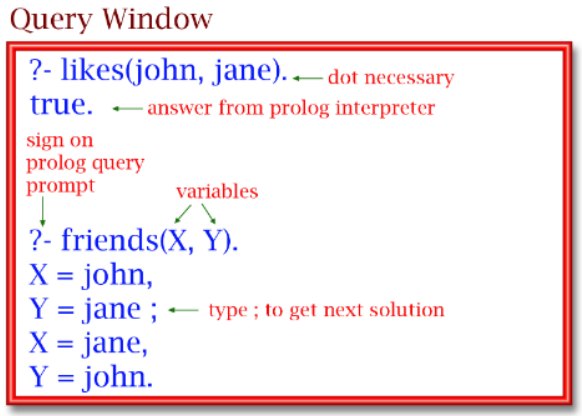


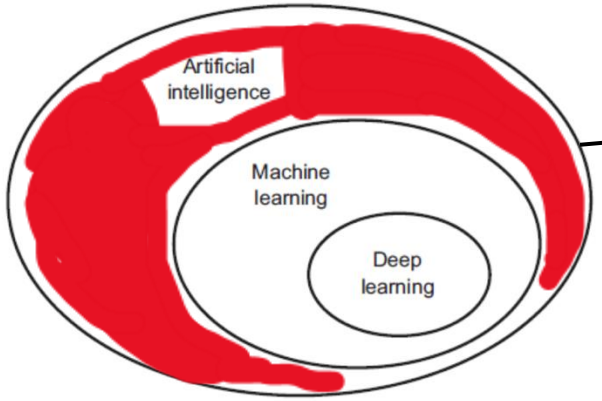
- rule-based systems that mimic human decision-making
- **Mycin**, an early AI system used for diagnosing bacterial infections and recommending antibiotics. It was based on predefined rules created by human experts, and the system would infer conclusions based on those rules. There is no learning involved—just following if-then logical rules.

Logic Programming:

- uses formal logic to express knowledge and solve problems by deriving logical conclusions
- In **Prolog**, a logic programming language, you can define facts and rules to model a system. The system uses logical inference to derive new facts from the rules. There is no learning from data involved—everything is explicitly programmed.



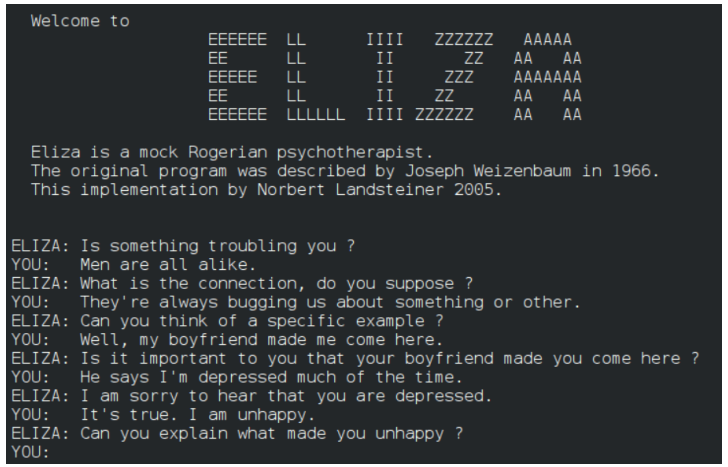




does not involve learning from data !

Example of problems:

Natural Language Processing (NLP) with Rule-Based Systems:

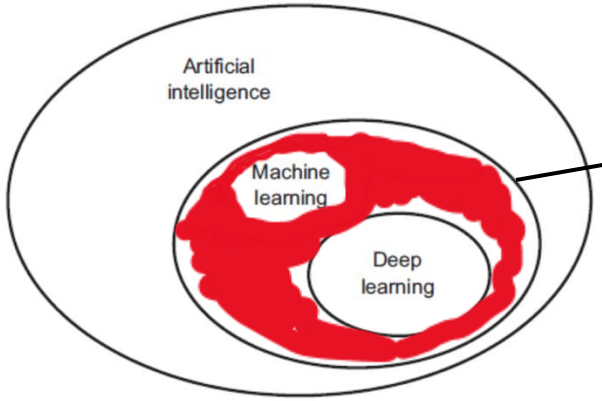


- Understand and process human language using predefined rules.
- Early chatbots like **ELIZA**, used simple pattern-matching rules to mimic human conversation. ELIZA did not learn from data but responded based on pre-programmed templates and keyword detection.
- ELIZA did not improve with more conversations. It simply followed its predefined logic and patterns.

Constraint Satisfaction Problems (CSPs):

- a type of problem where a solution must satisfy a set of constraints or limitations.
- CSPs don't learn from data; they solve by systematically checking possibilities under fixed rules
- **Sudoku** – each row, column, and 3x3 block must contain numbers 1–9 without repetition

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

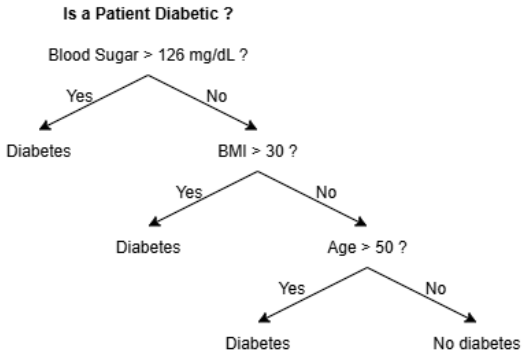


learn from data to make predictions or decisions!

Example of problems:

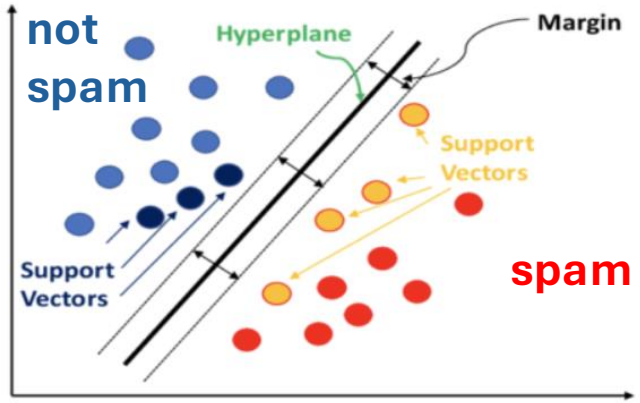
Decision Trees and Ensemble Methods (Random Forests, Gradient Boosting):

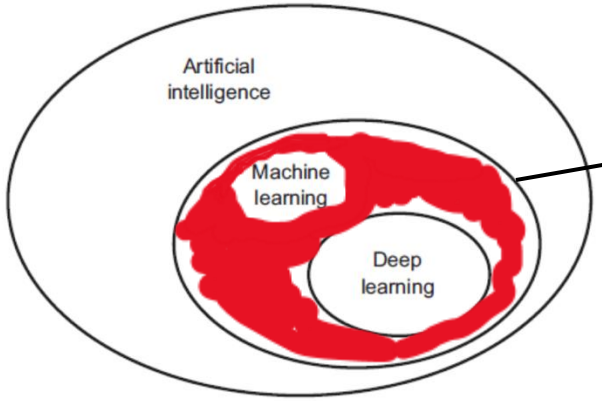
- Decision trees split data based on feature values to make predictions.
- Ensemble methods like Random Forests and Gradient Boosting combine multiple decision trees to improve accuracy and robustness.
- **Medical Diagnosis**: predict whether a patient has diabetes based on features like blood sugar level, BMI, and age.



Support Vector Machines (SVM):

- powerful classifiers that find the optimal hyperplane separating different classes in the feature space
- **Spam Detection**, where SVMs classify emails as spam or not spam based on features like keyword frequency and sender information.



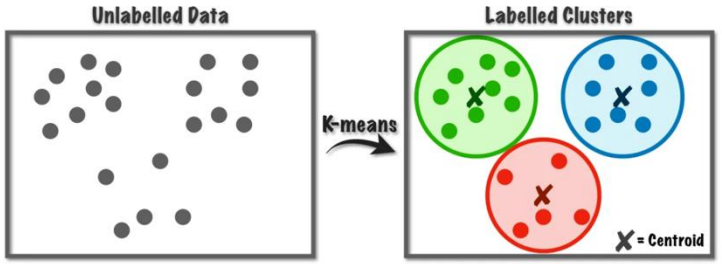


learn from data to make predictions or decisions!

Example of problems:

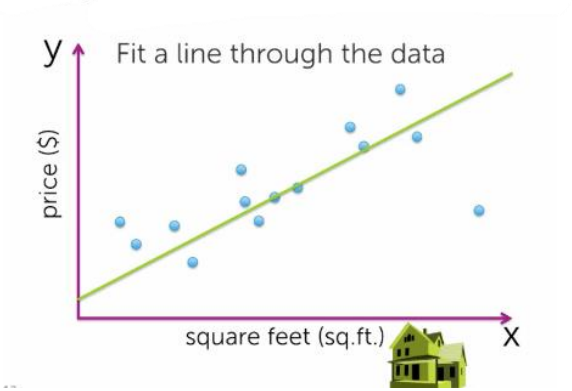
K-Means Clustering:

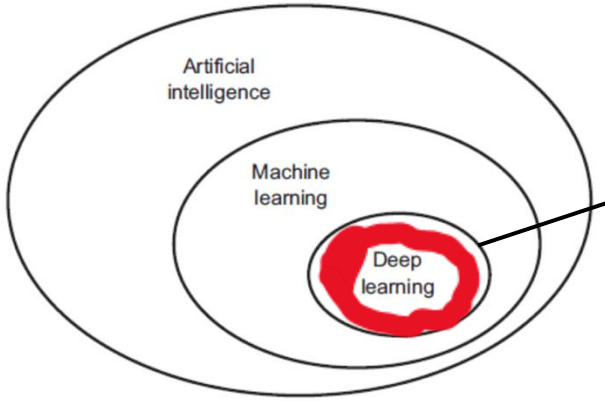
- a popular clustering algorithm that partitions data into K distinct groups based on feature similarity.
- **Customer Segmentation**: group customers based on purchasing patterns for targeted marketing.



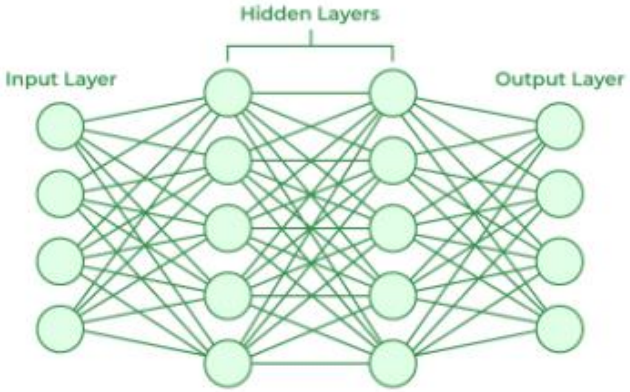
Linear and Logistic Regression:

- foundational ML techniques used for predicting continuous outcomes (linear regression) or categorical outcomes (logistic regression) based on input features.
- **House Price Prediction**, where linear regression models factors like size, location, and number of bedrooms to estimate property prices.



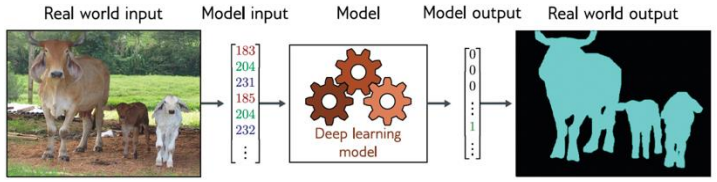


learn from data as in case of ML but works with (**deep**) neural networks:



Example of problems:

Image segmentation (semantic segmentation):



Depth estimation:

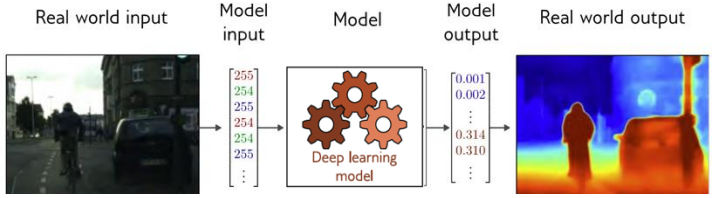
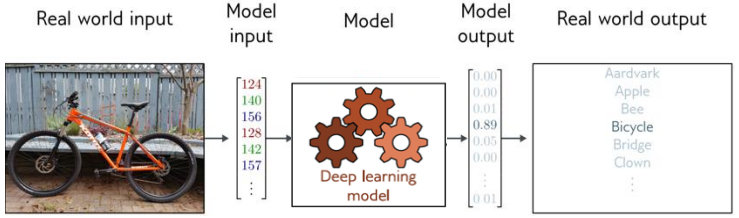
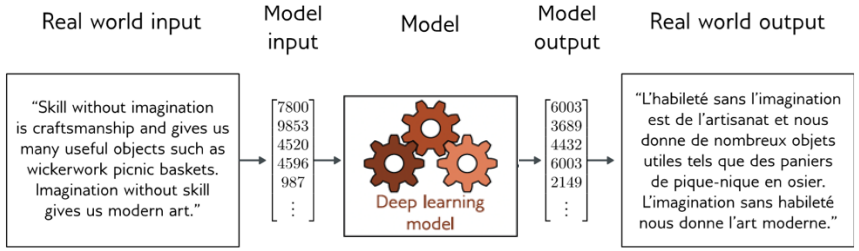


Image classification:

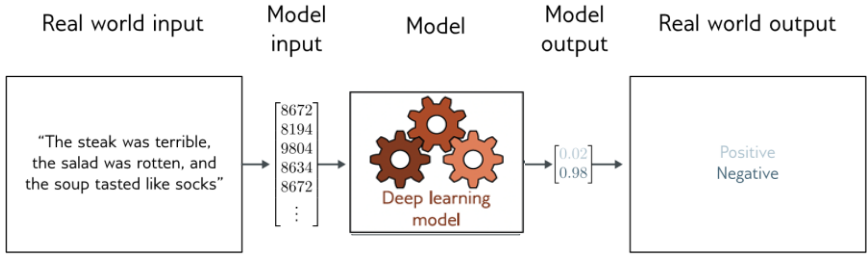


CV (computer vision) – works with images

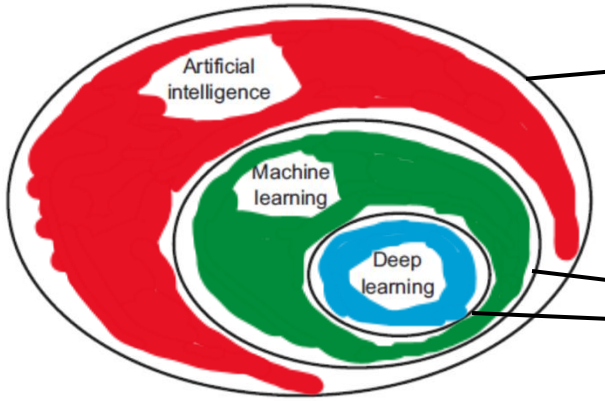
Translation:



Text classification:



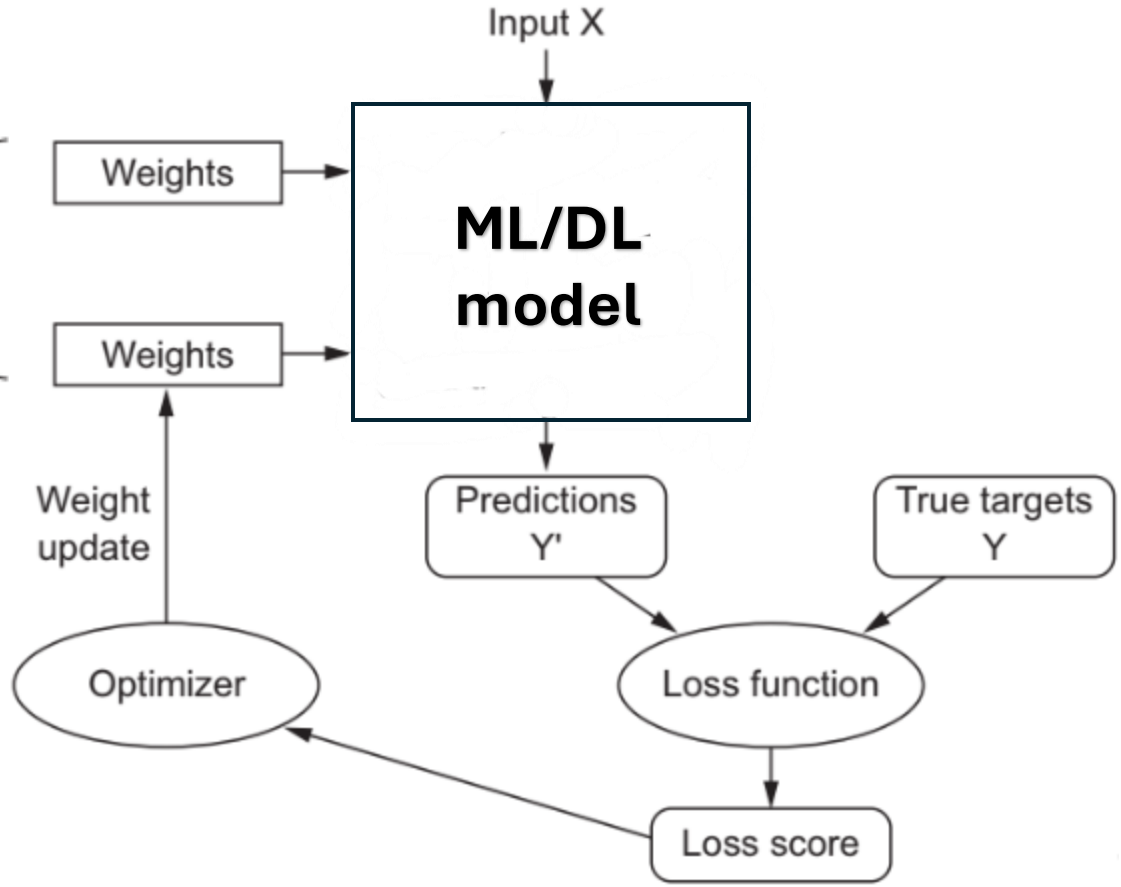
NLP (natural language processing) – works with text

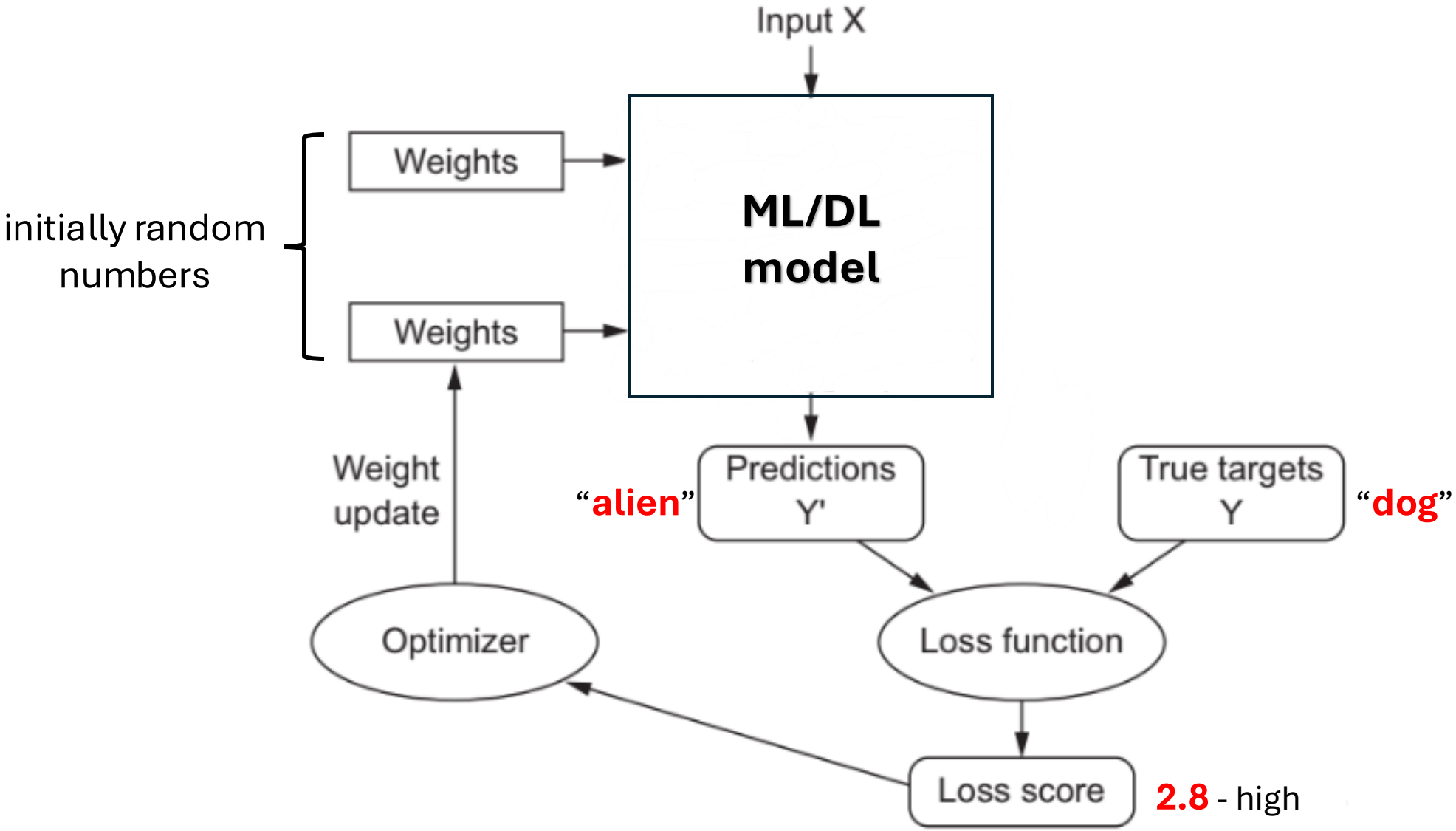


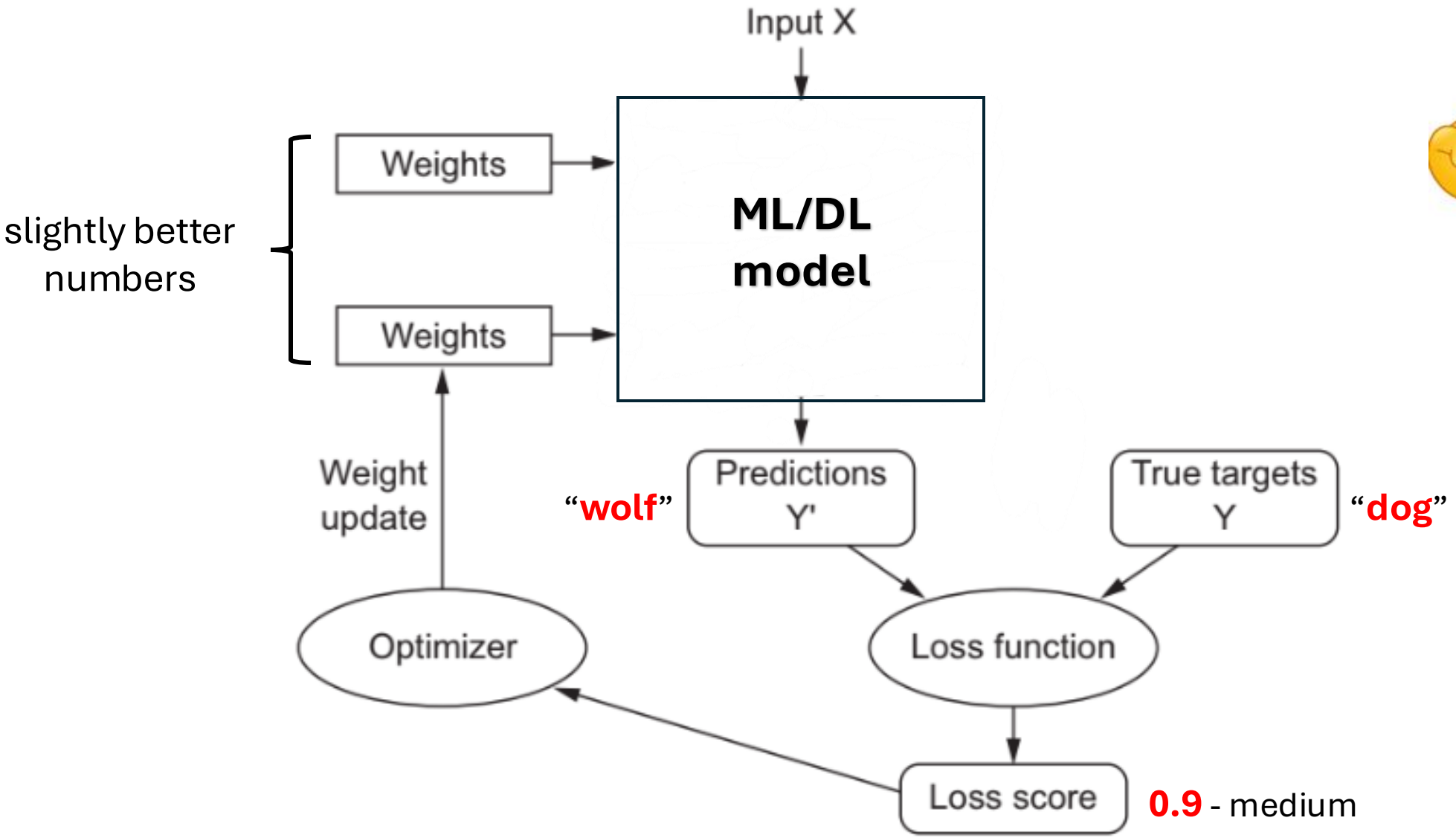
does not involve learning from data !

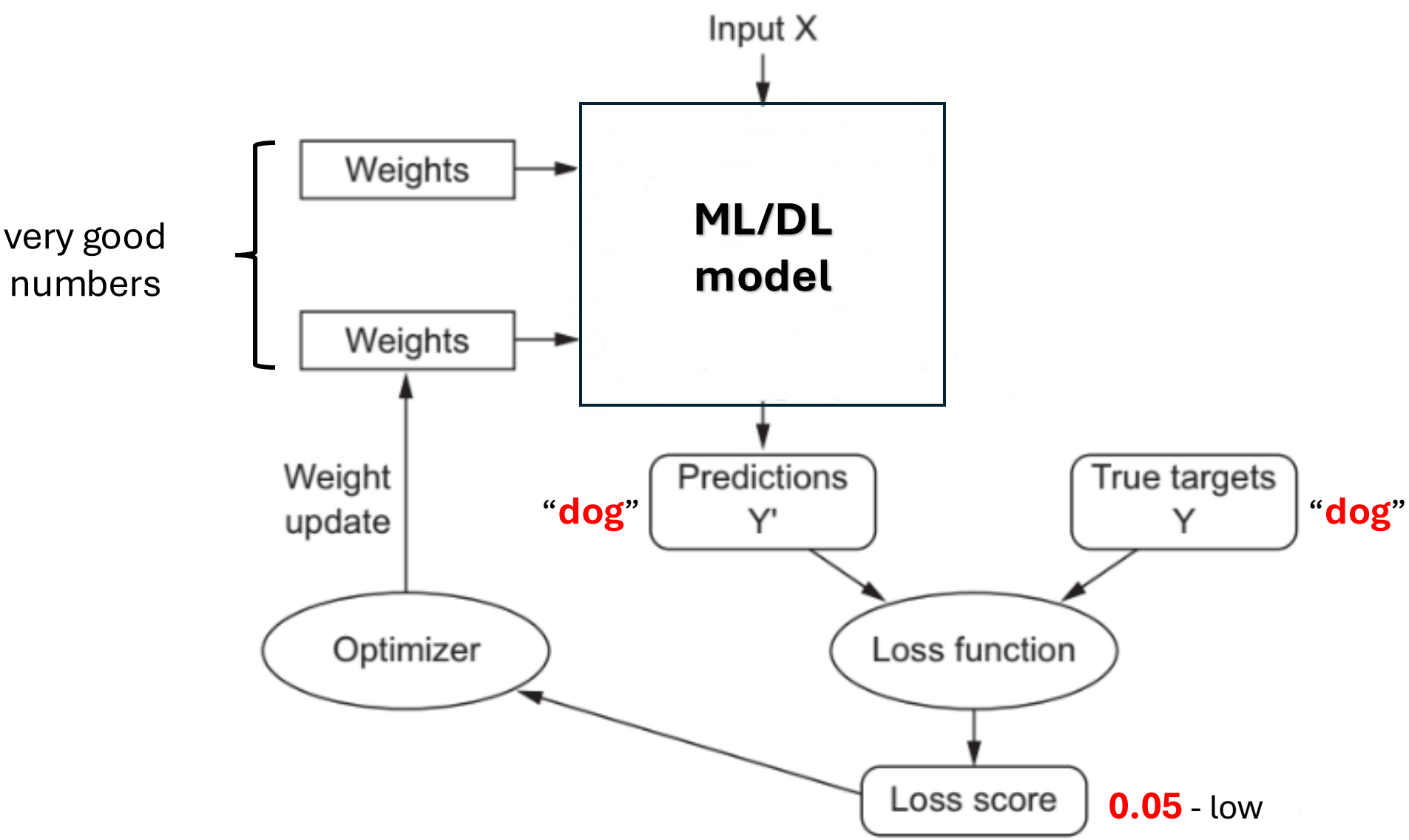
learn from data to make predictions or decisions!
(the models are “TRAINED”!)

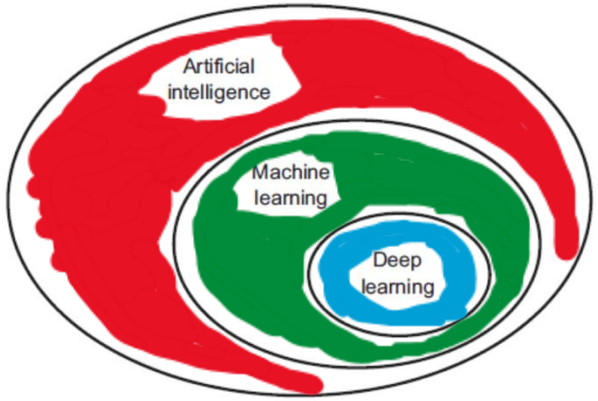
Goal: finding the right values for these weights




Input X
initially random numbers
Weights
Weights
ML/DL model
Weight update
"alien"
Predictions Y'
True targets Y
"dog"
Optimizer
Loss function
Loss score
2.8 - high


Input X
Weights
Weights
slightly better numbers
ML/DL model
Weight update
Predictions Y'
"wolf"
True targets Y
"dog"
Optimizer
Loss function
Loss score
0.9 - medium


Input X
very good numbers
Weights
Weights
ML/DL model
Weight update
"dog"
Predictions Y'
True targets Y
"dog"
Optimizer
Loss function
Loss score
0.05 - low



TL;DR — AI vs ML vs DL — Key Differences

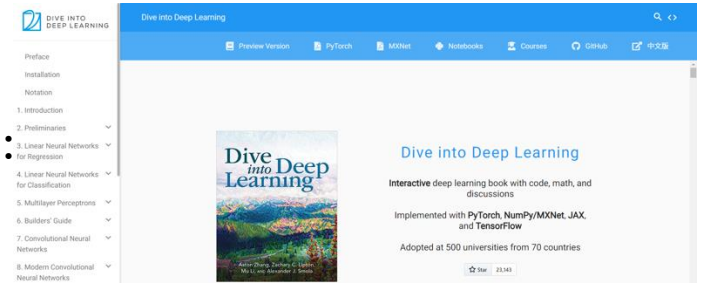
Artificial Intelligence (AI)
- Broad field: systems that **mimic human intelligence**.
- **Does not learn from data** (rule-based, expert systems, logic programming, CSPs).
- Examples**:** ELIZA chatbot, Sudoku solver, Mycin.

Machine Learning (ML)
- Subset of AI: **learns from data** to make predictions/decisions.
- Uses algorithms trained on data → models improve with experience.
- Examples**:** Decision Trees (medical diagnosis), SVMs (spam detection), Regression, Clustering.

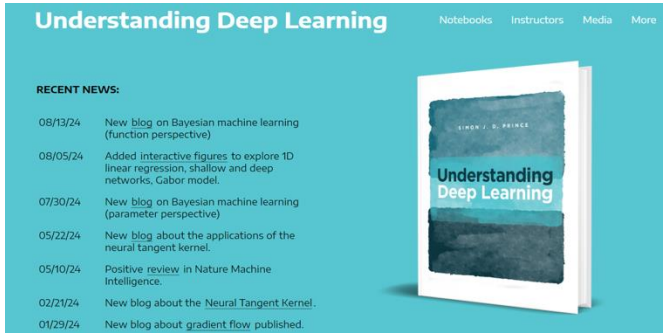
Deep Learning (DL)
- Subset of ML: uses **neural networks with many layers**.
- Learns features directly from raw data (images, text, audio).
- Examples**:** Image segmentation, Depth estimation, Translation, Sentiment analysis.

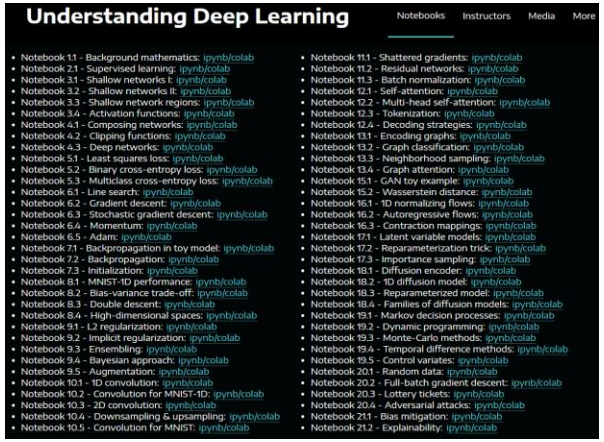
Some good resources to read and learn

d2l.ai:



https://udlbook.github.io/udlbook/ :



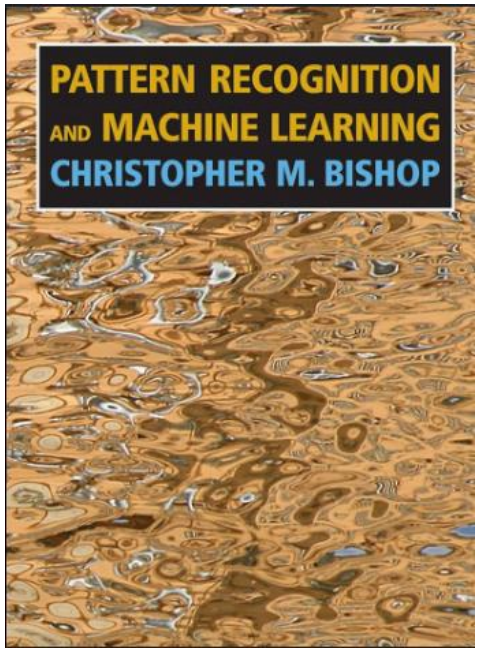


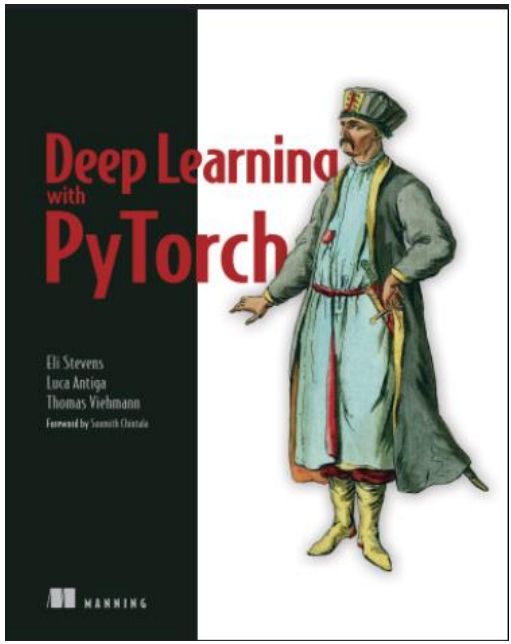
Andrew Ng courses and tutorials:

Jeremy Howard – Deep learning for Coders:

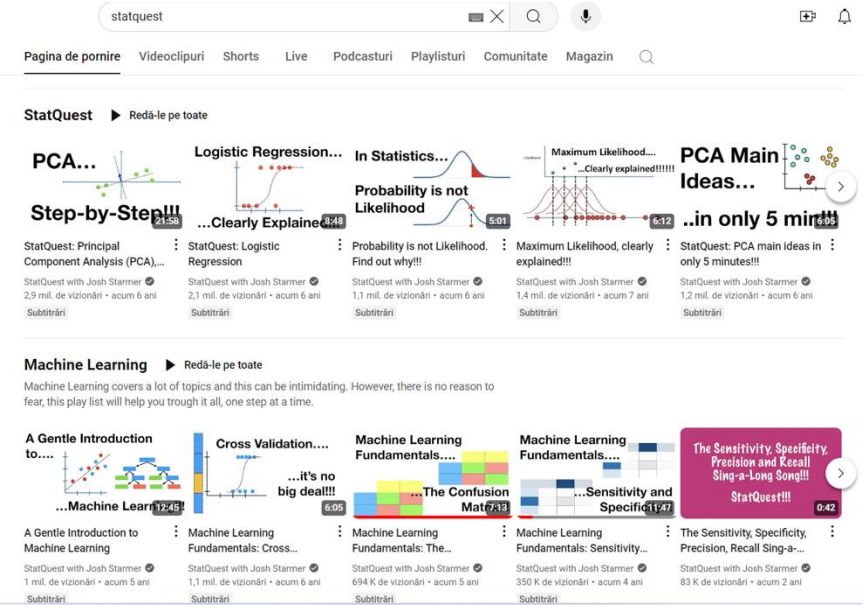
Some good resources to read and learn

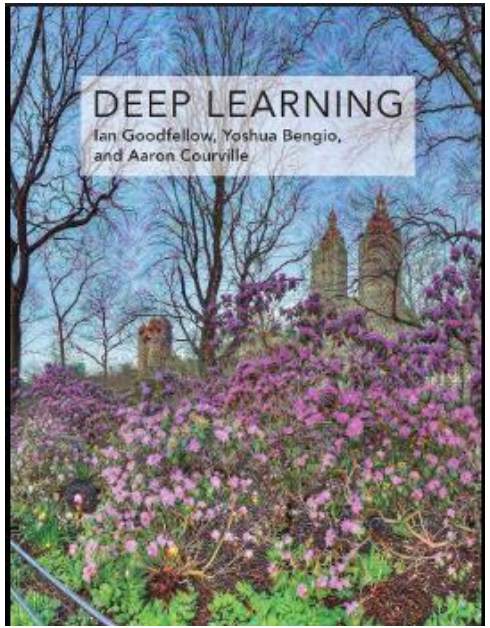
Andrej Karpathy and Fei-Fei Li courses:

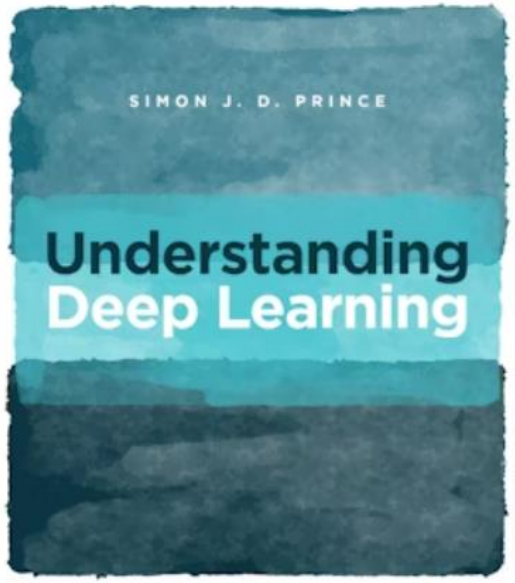




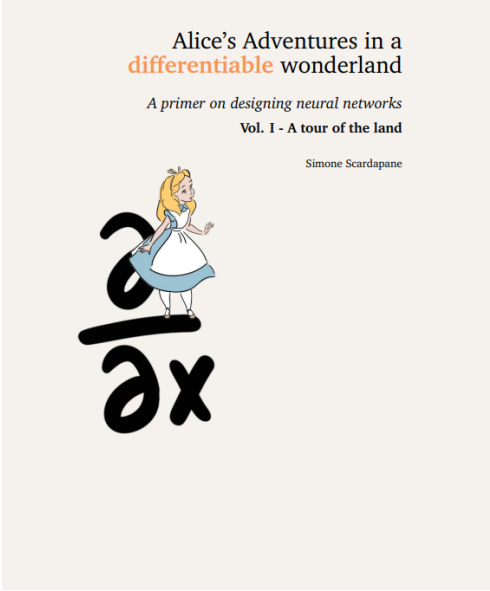
StatQuest playlists:

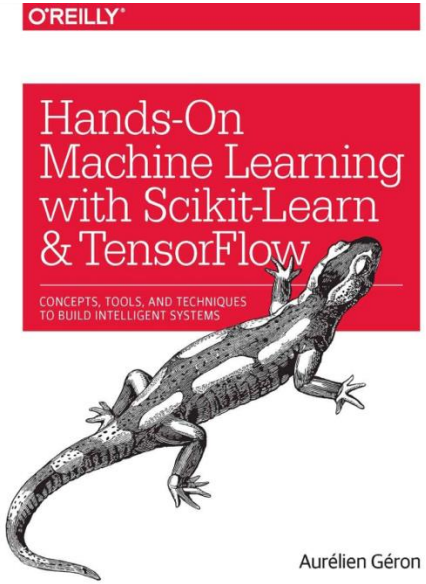


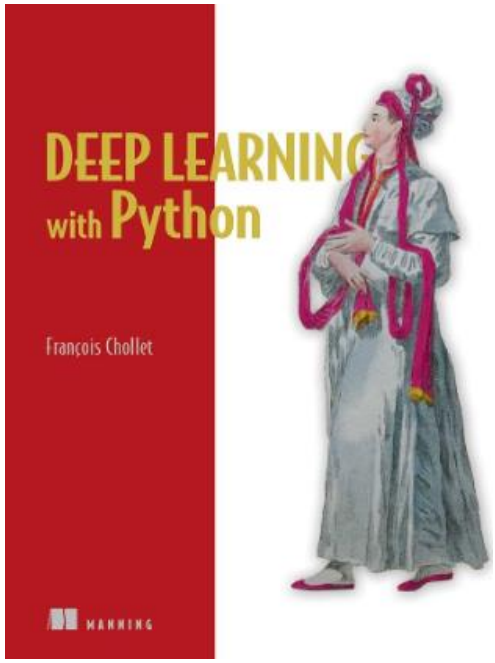


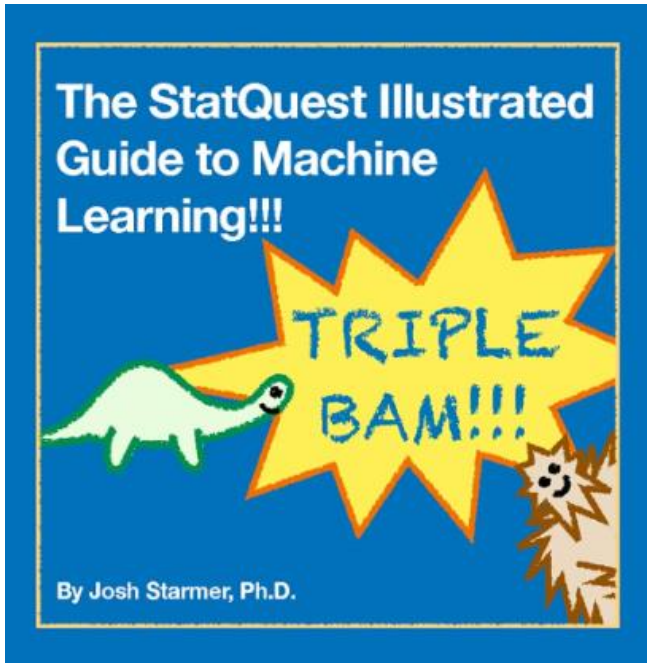


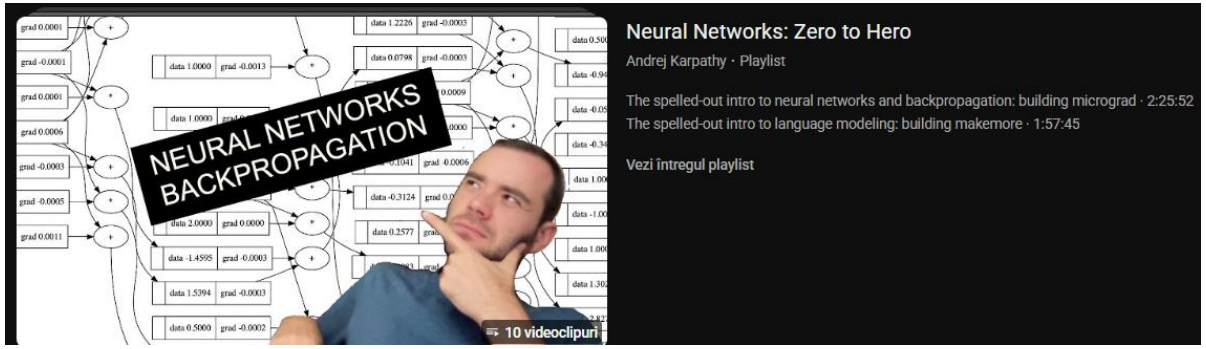
Some good resources to read and learn







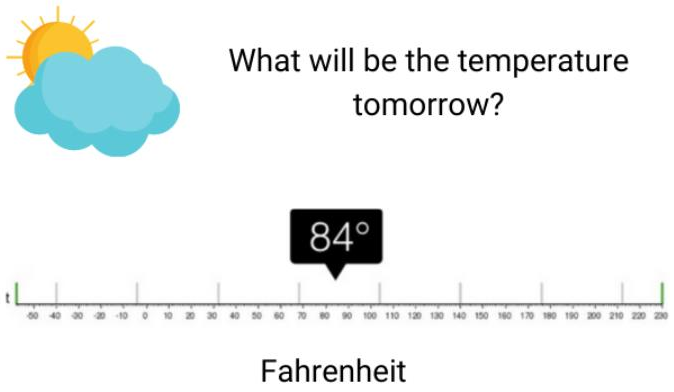




Some terminology…

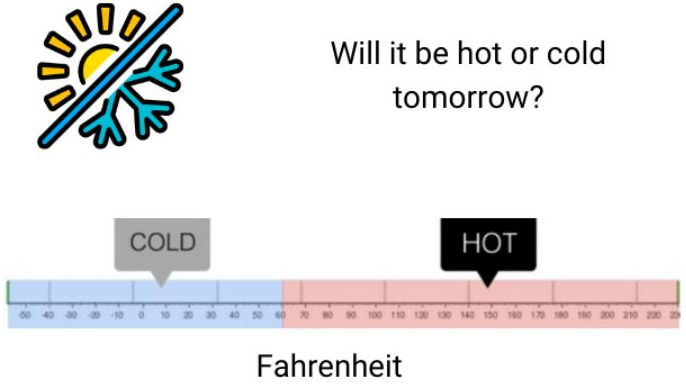
Regression

- aims at predicting a continuous numerical value based on input data.
- the output variable in regression is continuous, allowing for a range of possible values.
- **Example:** Predicting tomorrow's temperature (e.g., 84°F) based on factors like humidity, season, and weather patterns.



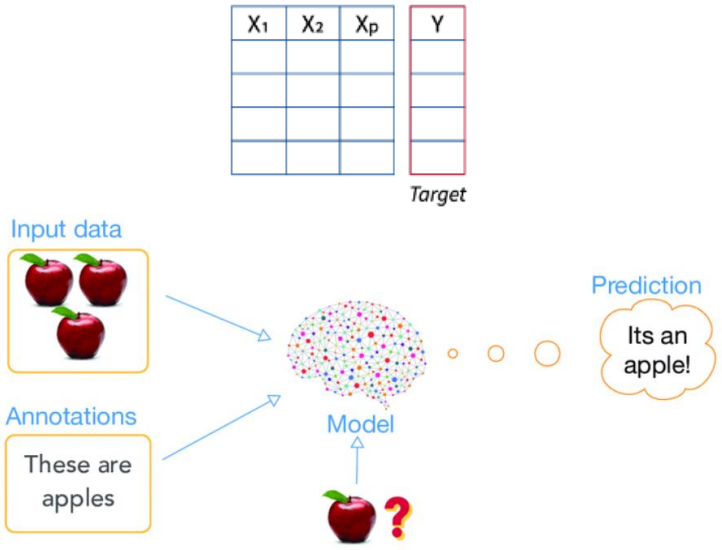
Classification

- the goal is to predict a discrete label or category for a given input data.
- the output variable in classification is categorical, meaning it represents distinct classes or groups.
- **Example:** Predicting whether tomorrow will be "hot" or "cold" based on today's weather conditions.



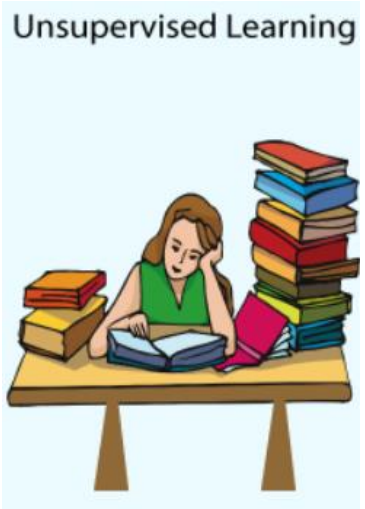
Supervised learning

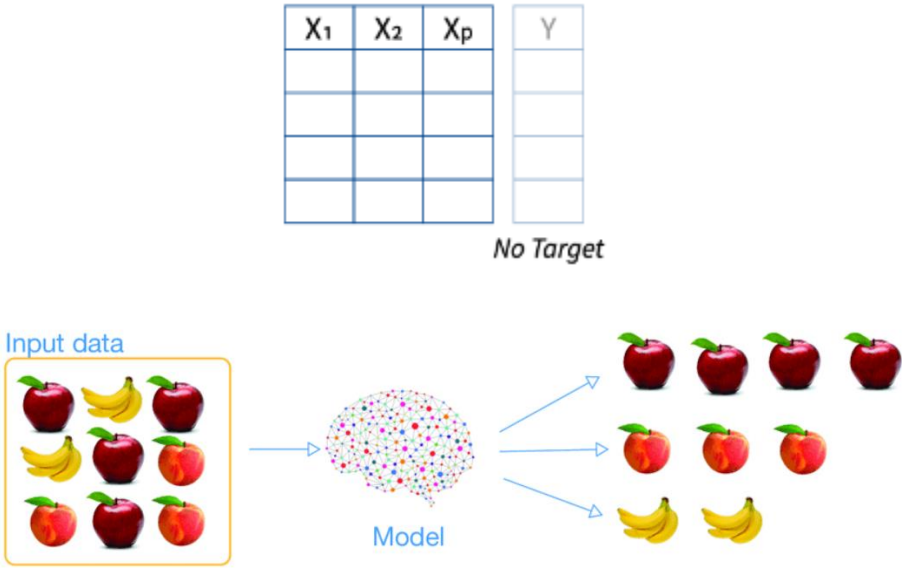
- Supervised learning involves training a model on a labeled dataset, where each training example is paired with an output label.
- The model learns to map inputs to the desired output based on these examples.
- **Common Algorithms**: Linear Regression, Logistic Regression, SVM, Decision Trees, Random Forests, Neural Networks (but not all !)



Unsupervised learning

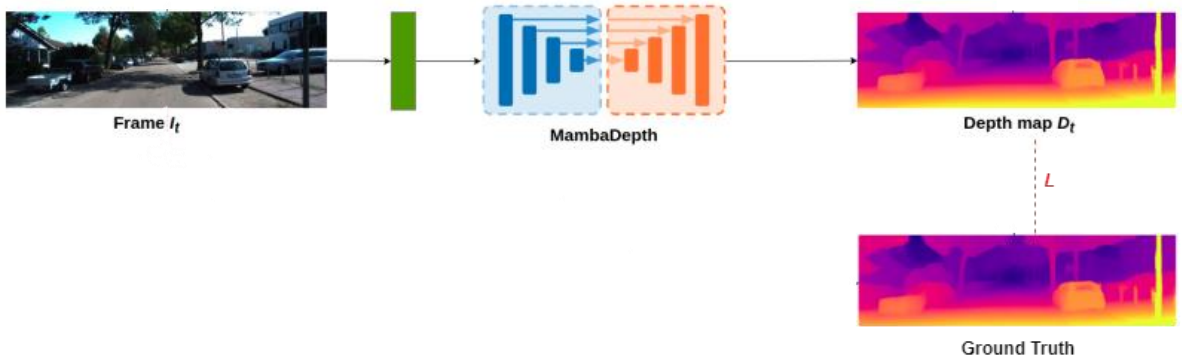
- Unsupervised learning deals with unlabeled data.
- The model tries to identify inherent patterns, structures, or relationships within the data without predefined labels.
- **Common Algorithms**: K-Means Clustering, Principal Component Analysis (PCA), Autoencoders, etc.





Supervised learning

Example: supervised depth estimation

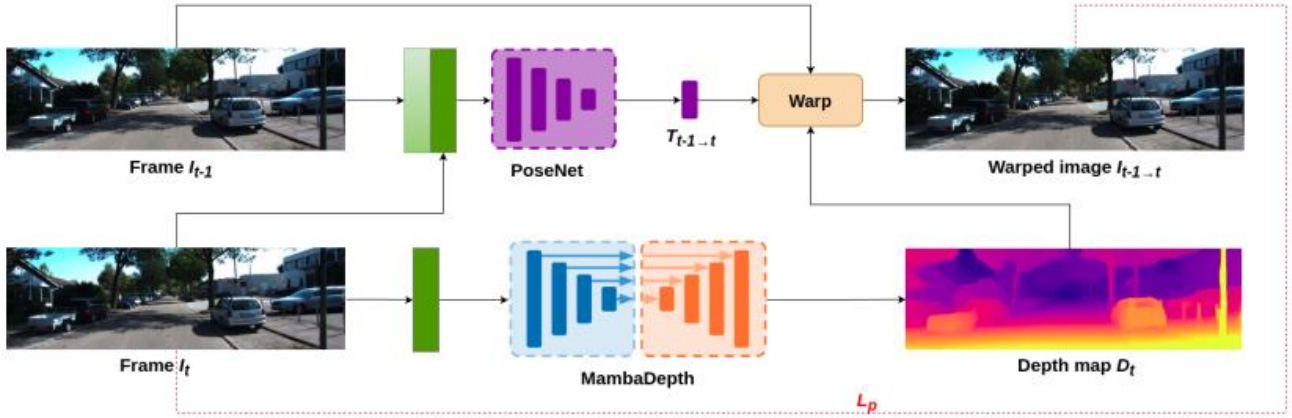


- Requires **Ground Truth (GT)** depth maps
- GT is usually obtained from **specialized sensors** (e.g., LiDAR, depth cameras), which are **expensive, slow, and limited**:

- Loss = difference between predicted depth and GT depth

Unsupervised learning

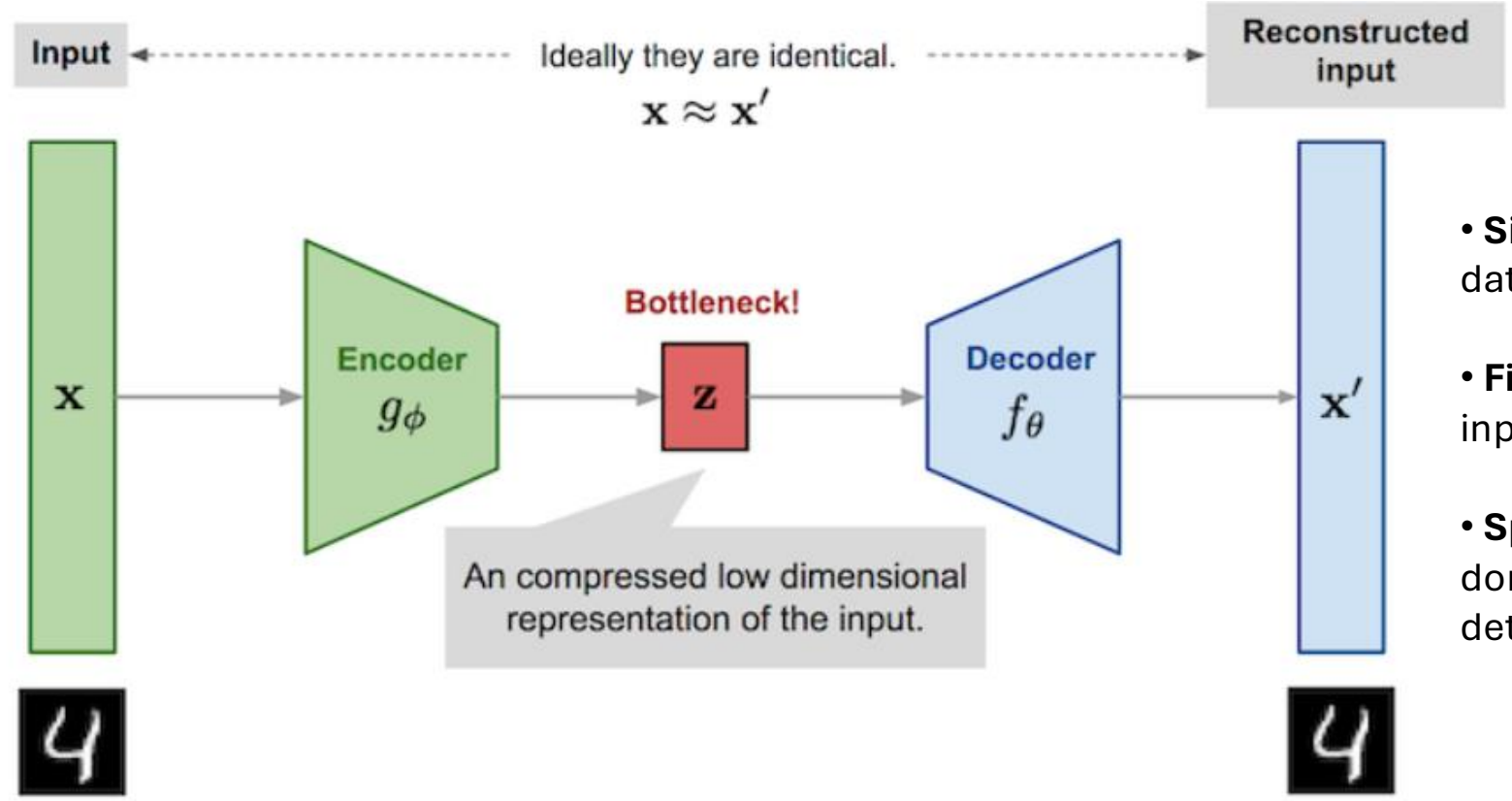
Example: unsupervised (self-supervised) depth estimation



- Requires only **videos**
- Depth is learned by reconstructing views across frames, no costly GT needed
- Loss = difference between real image and warped image (photometric error)

https://arxiv.org/pdf/2406.04532

...maybe a simpler example of unsupervised...



Autoencoder architecture

- **Simplifying Data**: It learns how to represent complex data in a simpler/compressed form.
- **Fixing Noisy Data**: It can clean up messy or blurry inputs (like images with noise).
- **Spotting Unusual Cases**: It helps detect things that don't fit the usual pattern (useful for fraud or error detection).

Laboratory contents

1. Introduction to Python → today's lab
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

Python Tutorial

Content

- Basic data types: *numbers*, *Booleans*, *strings*
- Containers: *lists*, *dictionaries*, *sets*, *tuples*
- Functions
- Classes
- Numpy
- Matplotlib

Basic data types

- Numbers
- Booleans
- Strings

```python
x = 3
print(type(x)) # Prints "<class 'int'>"
print(x)       # Prints "3"
print(x + 1)   # Addition; prints "4"
print(x - 1)   # Subtraction; prints "2"
print(x * 2)   # Multiplication; prints "6"
print(x ** 2)  # Exponentiation; prints "9"
x += 1
print(x)  # Prints "4"
```

```python
t = True
f = False
print(type(t)) # Prints "<class 'bool'>"
print(t and f) # Logical AND; prints "False"
print(t or f)  # Logical OR; prints "True"
print(not t)   # Logical NOT; prints "False"
```

```python
hello = 'hello'    # String literals can use single quotes
world = "world"    # or double quotes; it does not matter.
print(hello)       # Prints "hello"
print(len(hello))  # String length; prints "5"
hw = hello + ' ' + world  # String concatenation
print(hw)  # prints "hello world"
```

TAKEAWAYS:
1) Exponentiation (ridicarea la putere): x**2
2) To check the type: print(type(x))
3) Unlike many languages, Python does not have unary increment (x++) or decrement (x--) operator. Instead we use x+=1 or x-=1.
4) For Boolean logic, Python uses English words (and, or, not) rather than symbols (&&, ||, etc.)

Containers

- Lists
- Dictionaries
- Sets
- Tuples

1) Lists: the Python equivalent of arrays, but are resizeable and can contain elements of different types:

```
xs = [3, 1, 2]    # Create a list
print(xs[-1])     # Negative indices count from the end of the list; prints "2"
xs[2] = 'foo'     # Lists can contain elements of different types
print(xs)         # Prints "[3, 1, 'foo']"
xs.append('bar')  # Add a new element to the end of the list
print(xs)         # Prints "[3, 1, 'foo', 'bar']"
```

Containers

- Lists
- Dictionaries
- Sets
- Tuples

Slicing

```
nums = list(range(5))      # range is a built-in function that creates a list of integers
print(nums)                # Prints "[0, 1, 2, 3, 4]"
print(nums[2:4])           # Get a slice from index 2 to 4 (exclusive); prints "[2, 3]"
print(nums[2:])            # Get a slice from index 2 to the end; prints "[2, 3, 4]"
print(nums[:2])            # Get a slice from the start to index 2 (exclusive); prints "[0, 1]"
print(nums[:])             # Get a slice of the whole list; prints "[0, 1, 2, 3, 4]"
print(nums[:-1])           # Slice indices can be negative; prints "[0, 1, 2, 3]"
```

Loops

```
animals = ['cat', 'dog', 'monkey']
for animal in animals:
    print(animal)
```

```
cat
dog
monkey
```

or

```
animals = ['cat', 'dog', 'monkey']
for idx, animal in enumerate(animals):
    print('#%d: %s' % (idx + 1, animal))
```

```
#1: cat
#2: dog
#3: monkey
```

List comprehensions

```
nums = [0, 1, 2, 3, 4]
squares = [x ** 2 for x in nums]
print(squares)   # Prints [0, 1, 4, 9, 16]
```

Containers

- Lists
- Dictionaries
- Sets
- Tuples

2) Dictionaries: stores (key, value) pairs:

```
d = {'cat': 'cute', 'dog': 'furry'}  # Create a new dictionary with some data
print(d['cat'])       # Get an entry from a dictionary; prints "cute"
print('cat' in d)     # Check if a dictionary has a given key; prints "True"
d['fish'] = 'wet'     # Set an entry in a dictionary
print(d['fish'])      # Prints "wet"
# print(d['monkey'])  # KeyError: 'monkey' not a key of d
```

Containers

- Lists
- Dictionaries
- Sets
- Tuples

Loops

```
d = {'person': 2, 'cat': 4, 'spider': 8}
for animal in d:
    legs = d[animal]
    print('A %s has %d legs' % (animal, legs))
```

```
A person has 2 legs
A spider has 8 legs
A cat has 4 legs
```

or

```
d = {'person': 2, 'cat': 4, 'spider': 8}
for animal, legs in d.items():
    print('A %s has %d legs' % (animal, legs))
```

```
A person has 2 legs
A spider has 8 legs
A cat has 4 legs
```

Dictionary comprehensions

```
nums = [0, 1, 2, 3, 4]
even_num_to_square = {x: x ** 2 for x in nums if x % 2 == 0}
print(even_num_to_square)  # Prints "{0: 0, 2: 4, 4: 16}"
```

Containers

- Lists
- Dictionaries
- Sets
- Tuples

3) Sets: an unordered collection of distinct elements:

```
animals = {'cat', 'dog'}
print('cat' in animals)   # Check if an element is in a set; prints "True"
print('fish' in animals)  # prints "False"
animals.add('fish')       # Add an element to a set
print('fish' in animals)  # Prints "True"
print(len(animals))       # Number of elements in a set; prints "3"
```

Containers

- Lists
- Dictionaries
- Sets
- Tuples

Loops

```
animals = {'cat', 'dog', 'fish'}
for idx, animal in enumerate(animals):
    print('#%d: %s' % (idx + 1, animal))
```

```
#1: fish
#2: dog
#3: cat
```

NOTE: since sets are unordered, you cannot make assumptions about the order in which you visit the elements of the set

Set comprehensions

```
from math import sqrt
nums = {int(sqrt(x)) for x in range(30)}
print(nums)  # Prints "{0, 1, 2, 3, 4, 5}"
```

Containers

- Lists
- Dictionaries
- Sets
- Tuples

4) Tuples: an (immutable) ordered list of values:

```
d = {(x, x + 1): x for x in range(10)}  # Create a dictionary with tuple keys
t = (5, 6)        # Create a tuple
print(type(t))    # Prints "<class 'tuple'>"
print(d[t])       # Prints "5"
print(d[(1, 2)])  # Prints "1"
```

NOTE: **IMMUTABLE** means that once a tuple is created, its values **cannot be changed**. You cannot add, remove, or modify the elements in the tuple. This contrasts with lists, which are mutable (changeable).

Functions

Python functions are defined using the **def** keyword:

```
def sign(x):
    if x > 0:
        return 'positive'
    elif x < 0:
        return 'negative'
    else:
        return 'zero'

for x in [-1, 0, 1]:
    print(sign(x))
# Prints "negative", "zero", "positive"
```

optional keyword argument

```
def hello(name, loud=False):
    if loud:
        print('HELLO, %s!' % name.upper())
    else:
        print('Hello, %s' % name)

hello('Bob') # Prints "Hello, Bob"
hello('Fred', loud=True)  # Prints "HELLO, FRED!"
```

Classes

```python
class Greeter(object):

    # Constructor
    def __init__(self, name):
        self.name = name  # Create an instance variable

    # Instance method
    def greet(self, loud=False):
        if loud:
            print('HELLO, %s!' % self.name.upper())
        else:
            print('Hello, %s' % self.name)

g = Greeter('Fred')  # Construct an instance of the Greeter class
g.greet()            # Call an instance method; prints "Hello, Fred"
g.greet(loud=True)   # Call an instance method; prints "HELLO, FRED!"
```

Numpy

What is Numpy and why Numpy?

- is the core library for scientific computing in Python
- using Numpy instead of plain Python lists offers several advantages, especially when working with numerical data and performing mathematical computations:
    - **Efficient Memory Usage**
    - **Vectorized Operations**
    - **Optimized Libraries**

Numpy

- the core library for scientific computing in Python

Numpy arrays:

```
import numpy as np

a = np.array([1, 2, 3])   # Create a rank 1 array
print(type(a))            # Prints "<class 'numpy.ndarray'>"
print(a.shape)            # Prints "(3,)"
print(a[0], a[1], a[2])   # Prints "1 2 3"
a[0] = 5                  # Change an element of the array
print(a)                  # Prints "[5, 2, 3]"

b = np.array([[1,2,3],[4,5,6]])    # Create a rank 2 array
print(b.shape)                     # Prints "(2, 3)"
print(b[0, 0], b[0, 1], b[1, 0])   # Prints "1 2 4"
```

NOTE: The number of dimensions is the ***rank*** of the array; the ***shape*** of an array is a tuple of integers giving the size of the array along each dimension.

Numpy

- Numpy also provides many functions to create arrays:

```
import numpy as np

a = np.zeros((2,2))   # Create an array of all zeros
print(a)              # Prints "[[ 0.  0.]
                      #          [ 0.  0.]]"

b = np.ones((1,2))    # Create an array of all ones
print(b)              # Prints "[[ 1.  1.]]"

c = np.full((2,2), 7)  # Create a constant array
print(c)               # Prints "[[ 7.  7.]
                       #          [ 7.  7.]]"

d = np.eye(2)         # Create a 2x2 identity matrix
print(d)              # Prints "[[ 1.  0.]
                      #          [ 0.  1.]]"

e = np.random.random((2,2))  # Create an array filled with random values
print(e)                     # Might print "[[ 0.91940167  0.08143941]
                             #               [ 0.68744134  0.87236687]]"
```

Numpy

- Every numpy array is a grid of elements of the same type. Numpy provides a large set of numeric datatypes that you can use to construct arrays:

```
import numpy as np

x = np.array([1, 2])   # Let numpy choose the datatype
print(x.dtype)         # Prints "int64"

x = np.array([1.0, 2.0])   # Let numpy choose the datatype
print(x.dtype)             # Prints "float64"

x = np.array([1, 2], dtype=np.int64)   # Force a particular datatype
print(x.dtype)                         # Prints "int64"
```

Numpy

- **Slicing** and **indexing**:

```python
import numpy as np

# Create the following rank 2 array with shape (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Use slicing to pull out the subarray consisting of the first 2 rows
# and columns 1 and 2; b is the following array of shape (2, 2):
# [[2 3]
#  [6 7]]
b = a[:2, 1:3]
```

```python
# A slice of an array is a view into the same data, so modifying it
# will modify the original array.
print(a[0, 1])   # Prints "2"
b[0, 0] = 77     # b[0, 0] is the same piece of data as a[0, 1]
print(a[0, 1])   # Prints "77"
```

```python
import numpy as np

a = np.array([[1,2], [3, 4], [5, 6]])

bool_idx = (a > 2)   # Find the elements of a that are bigger than 2;
                     # this returns a numpy array of Booleans of the same
                     # shape as a, where each slot of bool_idx tells
                     # whether that element of a is > 2.

print(bool_idx)      # Prints "[[False False]
                     #          [ True  True]
                     #          [ True  True]]"

# We use boolean array indexing to construct a rank 1 array
# consisting of the elements of a corresponding to the True values
# of bool_idx
print(a[bool_idx])  # Prints "[3 4 5 6]"

# We can do all of the above in a single concise statement:
print(a[a > 2])     # Prints "[3 4 5 6]"
```

Numpy

- Basic mathematical functions:

```
import numpy as np

x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

# Elementwise sum; both produce the array
# [[ 6.0  8.0]
#  [10.0 12.0]]
print(x + y)
print(np.add(x, y))

# Elementwise difference; both produce the array
# [[-4.0 -4.0]
#  [-4.0 -4.0]]
print(x - y)
print(np.subtract(x, y))

# Elementwise product; both produce the array
# [[ 5.0 12.0]
#  [21.0 32.0]]
print(x * y)
print(np.multiply(x, y))

# Elementwise division; both produce the array
# [[ 0.2         0.33333333]
#  [ 0.42857143  0.5       ]]
print(x / y)
print(np.divide(x, y))

# Elementwise square root; produces the array
# [[ 1.          1.41421356]
#  [ 1.73205081  2.        ]]
print(np.sqrt(x))
```

- transposing operation:

```
import numpy as np

x = np.array([[1,2], [3,4]])
print(x)    # Prints "[[1 2]
            #          [3 4]]"
print(x.T)  # Prints "[[1 3]
            #          [2 4]]"
```

- ***sum*** operation:

```
import numpy as np

x = np.array([[1,2],[3,4]])

print(np.sum(x))  # Compute sum of all elements; prints "10"
print(np.sum(x, axis=0))  # Compute sum of each column; prints "[4 6]"
print(np.sum(x, axis=1))  # Compute sum of each row; prints "[3 7]"
```

NOTE: unlike MATLAB, * is elementwise multiplication, not matrix multiplication. We instead use the ***dot*** function to compute inner products of vectors, to multiply a vector by a matrix, and to multiply matrices.

```
import numpy as np

x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11, 12])

# Inner product of vectors; both produce 219
print(v.dot(w))
print(np.dot(v, w))

# Matrix / vector product; both produce the rank 1 array [29 67]
print(x.dot(v))
print(np.dot(x, v))

# Matrix / matrix product; both produce the rank 2 array
# [[19 22]
#  [43 50]]
print(x.dot(y))
print(np.dot(x, y))
```

Numpy

NOTE: unlike MATLAB, * is elementwise multiplication, not matrix multiplication. We instead use the ***dot*** function to compute inner products of vectors, to multiply a vector by a matrix, and to multiply matrices.

```python
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])
```

```python
result_elementwise = A * B
print(result_elementwise)
```

```
[[ 5 12]
 [21 32]]
```

```python
result_dot = np.dot(A, B)
print(result_dot)
```

```
[[19 22]
 [43 50]]
```

Numpy

- ***Broadcasting*** is a powerful mechanism that allows numpy to work with arrays of different shapes when performing arithmetic operations.
- Frequently we have a smaller array and a larger array, and we want to use the smaller array multiple times to perform some operation on the larger array:

```python
import numpy as np

# We will add the vector v to each row of the matrix x,
# storing the result in the matrix y
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
y = np.empty_like(x)   # Create an empty matrix with the same shape as x

# Add the vector v to each row of the matrix x with an explicit loop
for i in range(4):
    y[i, :] = x[i, :] + v

# Now y is the following
# [[ 2  2  4]
#  [ 5  5  7]
#  [ 8  8 10]
#  [11 11 13]]
print(y)
```

```python
import numpy as np

# We will add the vector v to each row of the matrix x,
# storing the result in the matrix y
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
vv = np.tile(v, (4, 1))   # Stack 4 copies of v on top of each other
print(vv)                 # Prints "[[1 0 1]
                          #          [1 0 1]
                          #          [1 0 1]
                          #          [1 0 1]]"
y = x + vv  # Add x and vv elementwise
print(y)  # Prints "[[ 2  2  4
          #          [ 5  5  7]
          #          [ 8  8 10]
          #          [11 11 13]]"
```

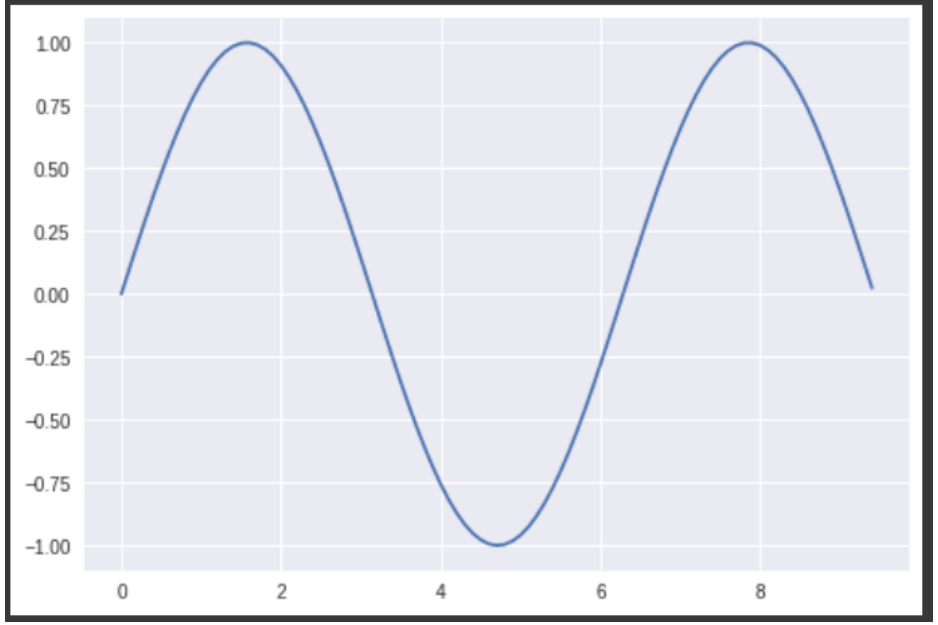
```python
import numpy as np

# We will add the vector v to each row of the matrix x,
# storing the result in the matrix y
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
y = x + v  # Add v to each row of x using broadcasting
print(y)  # Prints "[[ 2  2  4]
          #          [ 5  5  7]
          #          [ 8  8 10]
          #          [11 11 13]]"
```

All equivalent!

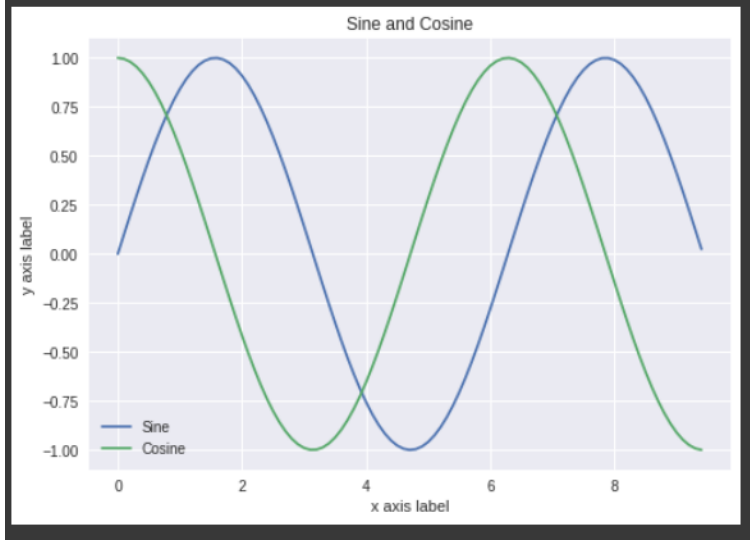
Matplotlib

- a plotting library
- **matplotlib.pyplot** module provides a plotting system similar to that of MATLAB.

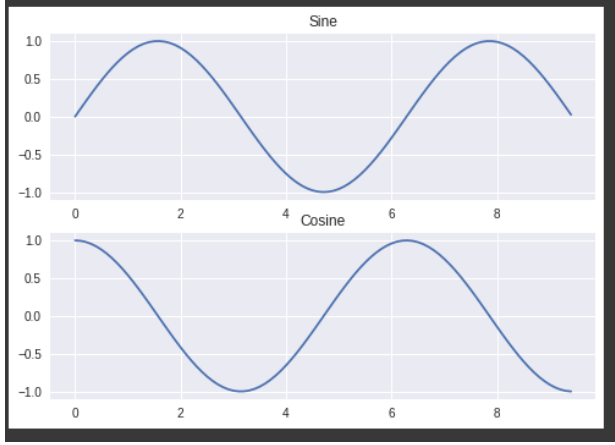
```python
import numpy as np
import matplotlib.pyplot as plt

# Compute the x and y coordinates for points on a sine curve
x = np.arange(0, 3 * np.pi, 0.1)
y = np.sin(x)

# Plot the points using matplotlib
plt.plot(x, y)
plt.show()  # You must call plt.show() to make graphics appear.
```




```python
import numpy as np
import matplotlib.pyplot as plt

# Compute the x and y coordinates for points on sine and cosine curves
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)

# Plot the points using matplotlib
plt.plot(x, y_sin)
plt.plot(x, y_cos)
plt.xlabel('x axis label')
plt.ylabel('y axis label')
plt.title('Sine and Cosine')
plt.legend(['Sine', 'Cosine'])
plt.show()
```




```python
import numpy as np
import matplotlib.pyplot as plt

# Compute the x and y coordinates for points on sine and cosine curves
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)

# Set up a subplot grid that has height 2 and width 1,
# and set the first such subplot as active.
plt.subplot(2, 1, 1)

# Make the first plot
plt.plot(x, y_sin)
plt.title('Sine')

# Set the second subplot as active, and make the second plot.
plt.subplot(2, 1, 2)
plt.plot(x, y_cos)
plt.title('Cosine')

# Show the figure.
plt.show()
```



How we will work at the laboratory?

We will use **Google Colab** (you need just a gmail to have the notebooks saved in Google Drive)

this is the laboratory from Campus

BIA_Laborator_1.ipynb

File Edit View Insert Runtime Tools Help Changes will not be saved

+ Code + Text Copy to Drive

Python Tutorial

Python

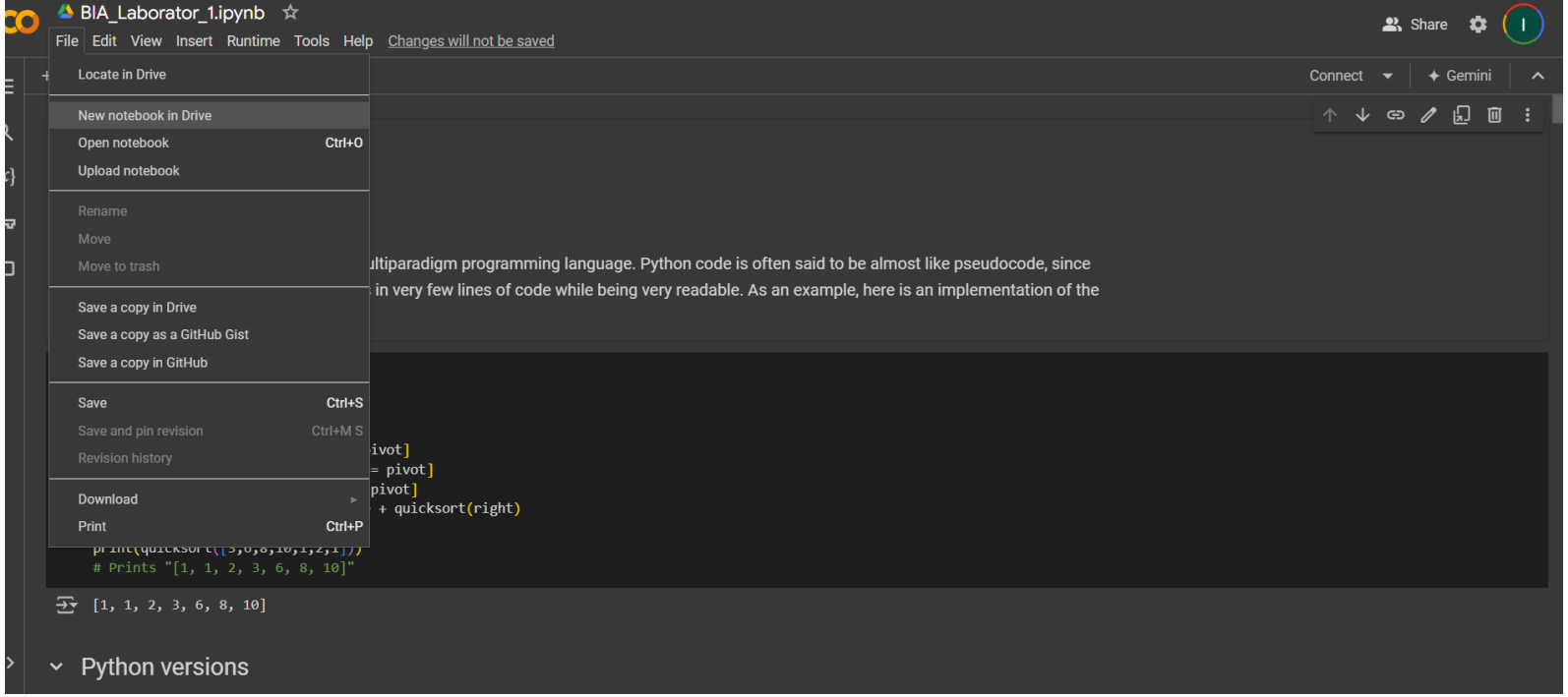
Python is a high-level, dynamically typed multiparadigm programming language. Python code is often said to be almost like pseudocode, since it allows you to express very powerful ideas in very few lines of code while being very readable. As an example, here is an implementation of the classic quicksort algorithm in Python:

```
[ ] def quicksort(arr):
        if len(arr) <= 1:
            return arr
        pivot = arr[len(arr) // 2]
        left = [x for x in arr if x < pivot]
        middle = [x for x in arr if x == pivot]
        right = [x for x in arr if x > pivot]
        return quicksort(left) + middle + quicksort(right)

    print(quicksort([3,6,8,10,1,2,1]))
    # Prints "[1, 1, 2, 3, 6, 8, 10]"
```

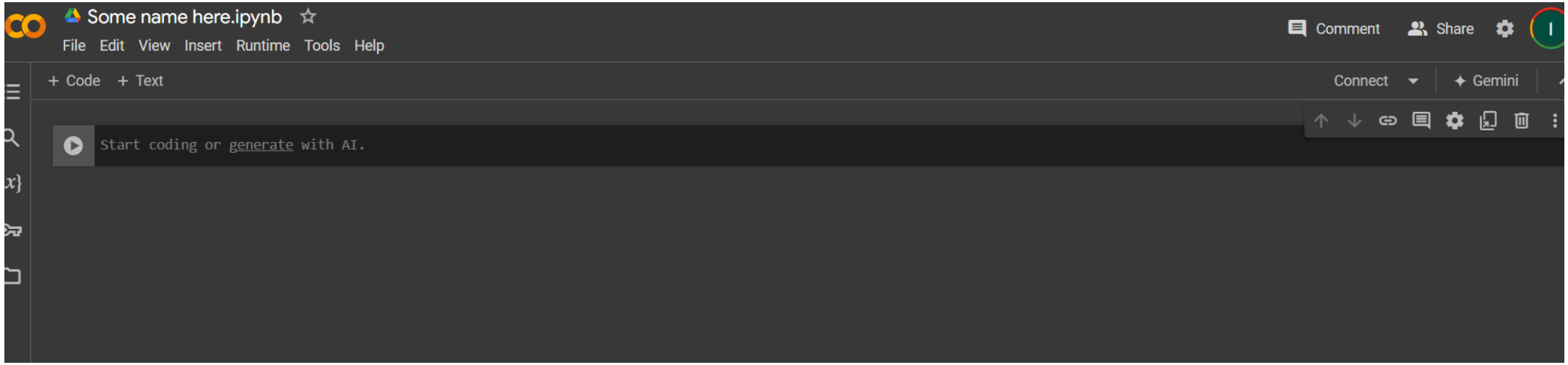
```
[1, 1, 2, 3, 6, 8, 10]
```

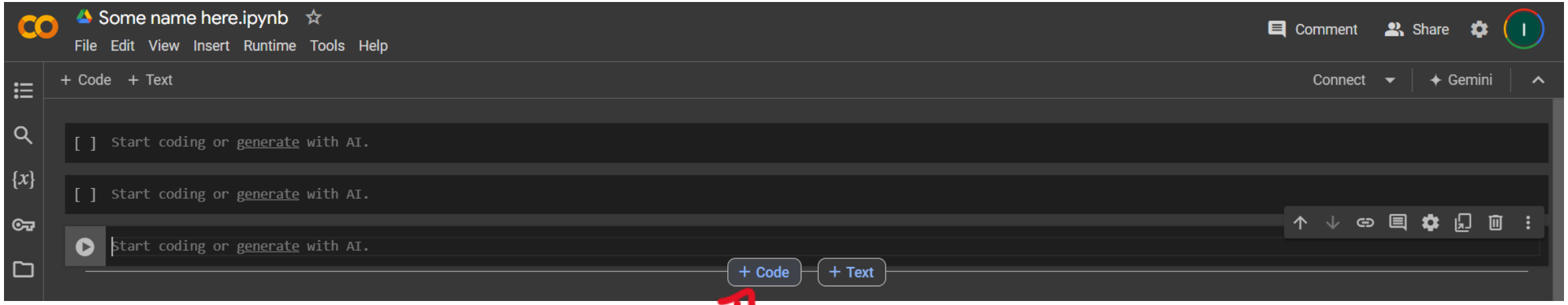
Python versions

There are currently two different supported versions of Python, 2.7 and 3.7. Somewhat confusingly, Python 3.0 introduced many backwards-incompatible changes to the language, so code written for 2.7 may not work under 3.7 and vice versa. For this laboratory all code will use Python 3.7

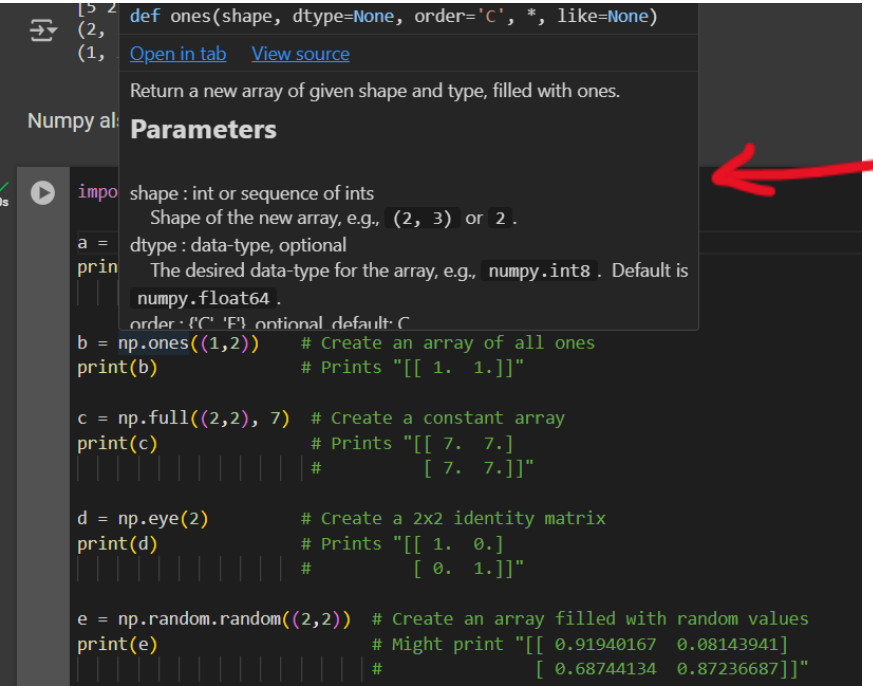


for laboratories and tests we will work in new notebooks





we can create any code cells we want (ideally it would be nice to have more cells for a single problem such that to be easier to debug the code ☺)



if we don't know exactly what a function does or what are its parameters, we can easily put the mouse over the name of the function, and then we will see a popup with a description for that function and some examples (if we scroll down a bit)

Exercises

Laboratory 1 – Exercises

1. Create a random NumPy array of five rows and four columns. Using array indexing and slicing, display the items from row three to end and column two to end.
2. An integer, *n*, is said to be *perfect* when the sum of all of the proper divisors of *n* is equal to *n*. For example, 28 is a perfect number because its proper divisors are 1, 2, 4 , 7 and 14, and 1+2+4+7+14=28.

   Write a function that determines whether or not a positive integer is perfect. Your function will take one parameter. If that parameter is a perfect number then your function will return True, otherwise it will return False.
3. Create a class with the following methods:
   - Constructor (__init__) – This method should take the argument *text,* make it lower case, and remove all punctuation. Assume only the following punctuation is used: period, exclamation mark, comma and question mark. Assign this newly formatted text to a new attribute called *fmtText*.
   - strLength – This method should return the length of the text.

Test your methods to ensure they are working correctly.

Solutions

Exercise 1

Create a random NumPy array of five rows and four columns. Using array indexing and slicing, display the items from row three to end and column two to end.

```
[ ] import numpy as np

    # Create a random numpy array of 5 rows and 4 columns
    array = np.random.rand(5, 4)

    # Use array indexing and slicing to display items from row three to the end and column two to the end
    sliced_array = array[2:, 1:]

    print(sliced_array)
```

```
[[0.03873654 0.82715774 0.65346454]
 [0.58311349 0.54868201 0.43174059]
 [0.92528032 0.79301696 0.12583047]]
```

Exercise 2

An integer, *n*, is said to be *perfect* when the sum of all of the proper divisors of *n* is equal to *n*. For example, 28 is a perfect number because its proper divisors are 1, 2, 4 , 7 and 14, and 1+2+4+7+14=28.

Write a function that determines whether or not a positive integer is perfect. Your function will take one parameter. If that parameter is a perfect number then your function will return True, otherwise it will return False.

```
[ ] def is_perfect_number(n):
        # Initialize sum of divisors
        divisors_sum = 0

        # Loop to find all divisors
        for i in range(1, n):
            if n % i == 0:
                divisors_sum += i

        # Check if the sum of divisors is equal to n
        return divisors_sum == n

    # Test the function with an example
    test_number = 28
    is_perfect = is_perfect_number(test_number)
    print(is_perfect)  # Output: True
```

```
True
```

Exercise 3

Create a class with the following methods:

- Constructor (__init__) – This method should take the argument *text,* make it lower case, and remove all punctuation. Assume only the following punctuation is used: period, exclamation mark, comma and question mark. Assign this newly formatted text to a new attribute called *fmtText*.
- strLength – This method should return the length of the text.

Test your methods to ensure they are working correctly.

```
[ ] class TextFormatter:
        def __init__(self, text):
            # Convert text to lower case
            text = text.lower()
            # Remove specified punctuation
            for punct in [".", "!", ",", "?"]:
                text = text.replace(punct, "")
            # Assign formatted text to attribute
            self.fmtText = text

        def strLength(self):
            # Return the length of the formatted text
            return len(self.fmtText)

    # Test the class
    test_text = "Hello, Students! How is the BIA laboratory today?"
    formatter = TextFormatter(test_text)

    # Test the methods
    formatted_text = formatter.fmtText
    length_of_text = formatter.strLength()

    print("Formatted Text:", formatted_text)
    print("Length of Text:", length_of_text)
```

```
Formatted Text: hello students how is the bia laboratory today
Length of Text: 46
```

Thank you !